Hotel Booking Cancellation Prediction & Overbooking Decision Support

Group 1 Final Project — Artificial Intelligence II

1. Problem Definition

Hotel revenue management faces a trade-off between maximizing occupancy and managing cancellation risk. Cancellations frequently leave rooms empty despite confirmed bookings, which is why overbooking is commonly used as an offset. However, overbooking introduces the opposite risk: if too many guests arrive, the hotel incurs costs from relocating customers and reputational damage.

This project focuses on supporting the booking acceptance decision under uncertainty. For each incoming reservation, the relevant question is whether the expected revenue from accepting justifies the overbooking risk, accounting for the probability that some existing bookings will cancel before arrival.

To address this, we model the probability of cancellation at the individual booking level, where the target variable is: **is_canceled = 1** - if the booking is cancelled / **is_canceled = 0** - if the guest arrives.

Prediction is useful because not all bookings carry the same cancellation risk. A guest who books months in advance through a third-party channel behaves very differently from one who books directly two days before arrival. A fixed overbooking rule ignores these differences and applies the same buffer regardless of who is actually booked on a given night.

By estimating per-booking cancellation probabilities, the system enables a more precise estimate of expected arrivals. These probabilities can then be aggregated to support overbooking decisions that are conditional on the current booking portfolio, rather than based on static rules.

https://github.com/jadzoghaib/Hotel_Booking_AI2

2. Dataset, EDA & Feature Engineering

Dataset Overview

The analysis uses the publicly available Hotel Booking Demand dataset, containing 119,390 reservations from a city hotel and a resort hotel spanning 2015 to 2017, anonymized and released for research purposes.

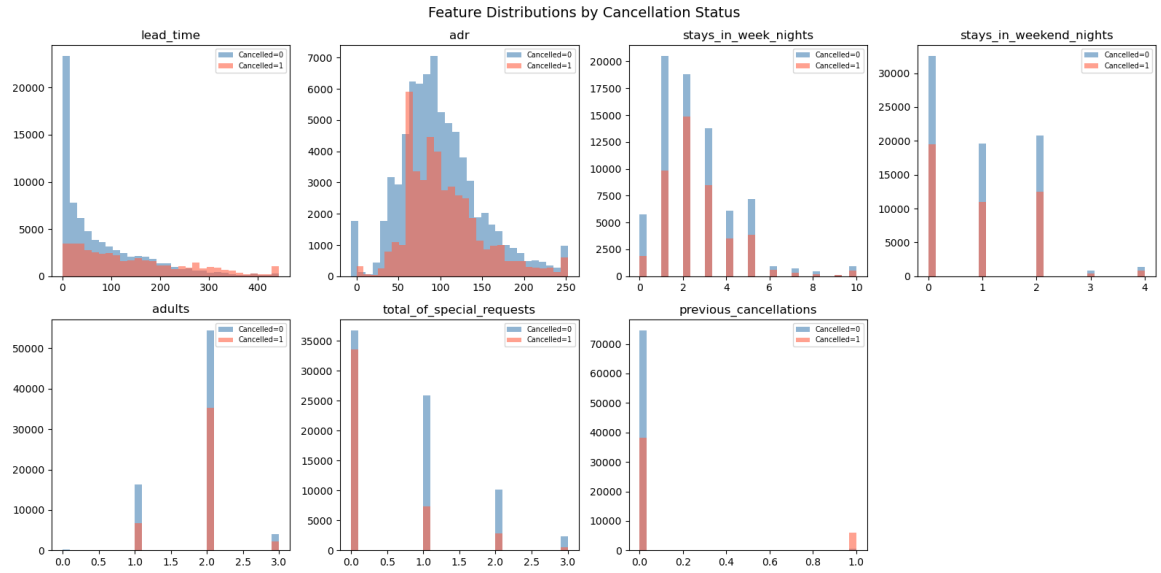
The dataset includes booking characteristics (e.g. lead time, room type, price), customer attributes, and distribution channel variables. The target variable is is_canceled, with an overall cancellation rate of 37%, resulting in moderate class imbalance.

Data Cleaning

Several variables were removed to prevent data leakage, as they are only available after the booking outcome (e.g. reservation_status, reservation_status_date, assigned_room_type). Missing values in agent and company were imputed as zero, interpreted as no intermediary involvement. A small number of incomplete rows were dropped to maintain consistency.

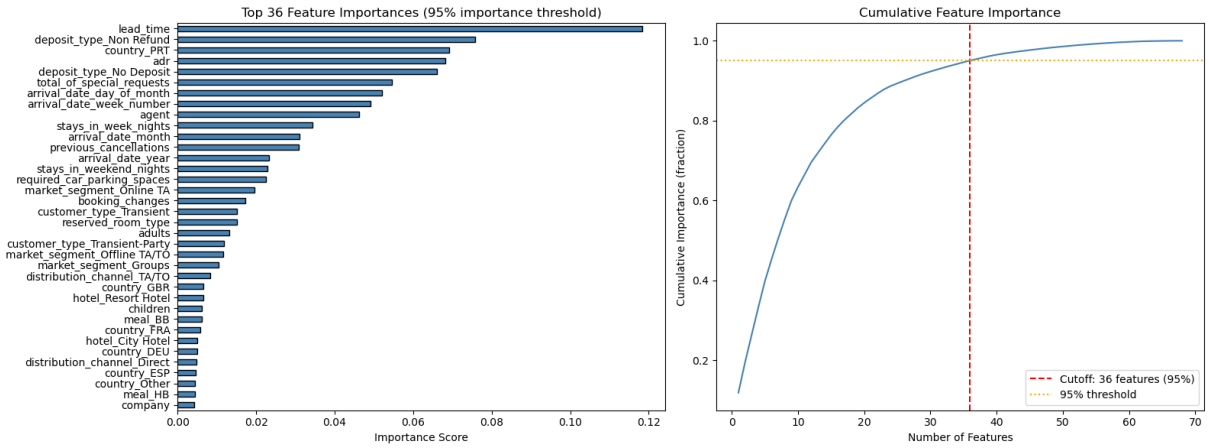
Exploratory Data Analysis

EDA highlights several systematic patterns in cancellation behaviour:



Feature Engineering & Selection

Categorical variables were encoded using ordinal encoding where order matters (room type, month) and one-hot encoding for nominal variables. Rare countries were grouped into "Other" to reduce sparsity, yielding 78 candidate features. A Random Forest was then used to rank features by Gini importance, and a 95% cumulative importance threshold was applied, reducing the final set to 36 features. 'hotel_City Hotel' was manually force-included to preserve hotel-type information regardless of rank. This feature selection approach limits overfitting and keeps computational cost manageable at scale.



3. Model Implementation, Evaluation & Decision System

Model Choice & Justification

Six classifiers were evaluated using a consistent 80/20 stratified train–test split:

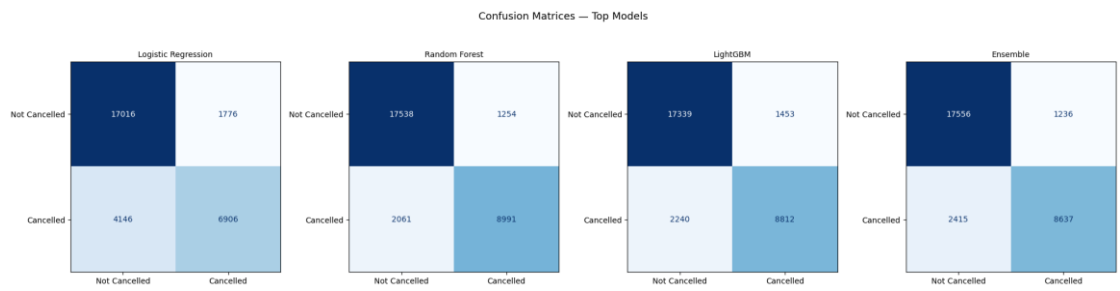
- Logistic Regression: interpretable baseline for comparisons
- Random Forest: handles the mixed feature types and non-linear interactions in our data without extra engineering. Robust to class imbalance and outputs the per-booking probabilities required by the Monte Carlo engine that we implemented in our next step
- XGBoost: gradient boosting for the structured tabular data with layered interactions (like lead time, deposit type and market segment)
- LightGBM: same boosting logic as XGBoost but trains significantly faster. Especially that since in practice the model would be periodically trained to adjust to trends
- Neural Network (MLP): tests whether a deep architecture captures high-order feature interactions that tree models miss
- Soft Voting Ensemble: averages the probability outputs of the four non-linear models to reduce variance. Included to verify whether combining predictions outperforms the single best model

Model performance was further validated using 5-fold stratified cross-validation to assess generalization stability.

Model Performance Summary

Model	Accuracy	Precision	Recall	F1	Test AUC	CV AUC (±SD)
Logistic Regression	0.805	0.803	0.628	0.705	0.885	0.886 ±0.001
Random Forest (tuned) ★	**0.889**	**0.879**	**0.811**	**0.844**	**0.958**	**0.960 ±0.001**
XGBoost (tuned)	0.879	0.857	0.807	0.831	0.952	0.944 ±0.001
LightGBM	0.877	0.859	0.798	0.827	0.951	0.947 ±0.001
MLP (Neural Network)	0.863	0.846	0.771	0.807	0.939	0.934 ±0.001
Soft-Voting Ensemble	0.878	0.876	0.780	0.826	0.951	—

Table 1. Model performance on the 25% held-out test set. ***Random Forest*** *selected as production model (highest AUC & F1). CV AUC computed with 5-fold stratified cross-validation.*



Confusion matrices for the four tree-based models on the held-out test set.

Other ensemble methods (XGBoost, LightGBM) perform similarly but slightly below Random Forest, while Logistic Regression and MLP show lower performance, suggesting that non-linear interactions are important in this setting.

4. Decision System

Stochastic Problem Framing

The number of guests who show up on a given night is a random variable. Even with a full booking list, arrival count is uncertain because each booking cancels independently with a different probability.

Formal setup:

- Let N = number of confirmed bookings
- Let p_i = ML-predicted cancellation probability for booking i
- Each booking's show-up indicator: $X_i \sim \text{Bernoulli}(1 - p_i)$
- Total arrivals: $A = \Sigma X_i$

Because each X_i has a different parameter, the sum has no closed-form distribution — simulation is the natural solution.

Monte Carlo Implementation

How the simulation runs:

- For each of the N confirmed bookings, sample $X_i \sim \text{Bernoulli}(1 - p_i)$
- Sum the results to get one realised arrival count
- Repeat 10,000 times to build an empirical arrival distribution
- Read P(A > capacity) directly from that distribution

Acceptance logic:

When a new booking arrives, the risk P(A > capacity) is computed before and after tentatively adding it

If the posterior risk stays below the configured threshold (default 5%), the booking is accepted

Pipeline summary:

Preprocessing → Random Forest scoring → Monte Carlo simulation → accept/reject decision

Streamlit demo of the approach: https://jadzoghaib-hotel-booking-ai2-app-my01c0.streamlit.app/

Critical Reflection

Perceived limitations

Incomplete cancellation definition: is_canceled excludes no-shows, so the model underestimates true arrival risk, which directly impacts overbooking decisions.

Data staleness: training data is from 2015–2017. As cancellation behaviour has shifts (especially post-pandemic) the model will misestimate risk until retrained. As such, the model needs to be retrained periodically on more recent data which could be computationally expensive without adequate resources.

Proxy bias: country of origin may proxy for demographic characteristics rather than booking intent, introducing biased predictions across customer groups.

Project reflection

Our initial project was more ambitious. We tried to build a real-time natural disaster cost estimator and decision-support tool for project prioritization (https://github.com/jadzoghaib/Disaster_Classification_FEMA). The model performance showed limited value-created for decision makers, despite our attempts to enrich it through external APIs and meteorological datasets. Despite extensive data wrangling, we saw no meaningful improvement in model performance, preventing us from surpassing baseline results reported in the academic research.

At some point we had to make a call: keep pushing on a problem where the data was fundamentally limiting us, or pivot to something where we could actually provide actionable insights. We chose the latter.

The first project was not wasted time. It taught us that the gap between an interesting problem and a tractable one is wider than we initially assumed. A well-scoped project with a strong dataset will produce better work than an ambitious one built on shaky foundations, and we had to learn that the hard way.